

keyle

v0.4.0

This package provides a simple way to style keyboard shortcuts in your documentation.

<https://github.com/wesleyel/keyle> MIT License

Ctrl + Shift + P

Contents

1	About	2
2	Usage	2
2.1	Importing	2
2.2	Quick Start	2
2.3	Shortcut Strings	2
2.4	Key Aliases and the Mac Layout	3
2.5	HTML Export	3
3	Themes	3
3.1	Built-in Themes	3
3.2	Custom Themes	3
3.3	Extending a Theme with <code>.with()</code>	4
4	Symbols	4
4.1	Mac Keyboard Symbols	4
4.2	Linux Biolum Keyboard Symbols	5
4.3	SVG Key Glyphs	6
5	Reference	6
5.1	keyle	6
5.2	cap	8

1 About

KEYLE is a library that allows you to create HTML `<kbd>` like keyboard shortcuts simple and easy.

The name, `keyle`, is a combination of `key` and `theme`.

This project is inspired by [auth0/kbd](#) and [dogezzen/badgery](#). Send them respect!

2 Usage

2.1 Importing

KEYLE is imported using

```
1 #import "@preview/keyle:0.4.0"
```

2.2 Quick Start

The default renderer works out of the box:

```
1 #import "@preview/keyle:0.4.0": kbd
```

```
1 #kbd("Ctrl", "Shift", "P")
2
3 #kbd("Ctrl+Shift+P")
```

Ctrl + Shift + P

Ctrl + Shift + P

For customization, generate your own renderer with the `config` function (see Section 5). Section 3.1 lists available themes; a theme can be given by name or as a function.

```
1 #let kbd = keyle.config(theme: "flowbite")
2 #kbd("Ctrl", "Shift", "Alt", "Del")
3
4 #let kbd = keyle.config(
5   theme: keyle.themes.bioluminum,
6   delim: keyle.bioluminum-key.delim_plus
7 )
8 #kbd("Ctrl", "Shift", "Alt", "Del")
```

Ctrl + Shift + Alt + Del

Ctrl + ⇧ + Alt + Del

There are `compact` and `delim` options to make the output more compact and change the delimiter between keys.

You can either use them in the `config` function or directly in the generated `kbd` function.

```
1 #let kbd = keyle.config(compact: true, delim: "-")
2 #kbd("Ctrl", "Shift", "Alt", "Del")
3
4 #let kbd = keyle.config()
5 #kbd("Ctrl", "Shift", "Alt", "Del", compact: true,
6   delim: "-")
```

Ctrl-Shift-Alt-Del

Ctrl-Shift-Alt-Del

2.3 Shortcut Strings

Strings containing `+` are split into individual keys, so a whole shortcut can be pasted as one argument. A lone `+` (or a string where splitting would produce an empty part) is kept literal – pass a plus key as its own argument. Set `parse: false` in `config` to disable splitting entirely.

```
1 #kbd("Ctrl+Shift+P") \
2 #kbd("Ctrl", "+") #kbd("+")
```

Ctrl + Shift + P

Ctrl + + +

2.4 Key Aliases and the Mac Layout

Key names are normalized case-insensitively through an alias table. By default only unambiguous names are mapped (`cmd` / `command` → `⌘`, `opt` / `option` → `⌥`, `up` / `down` / `left` / `right` → arrows), so `Ctrl`, `Alt` and friends render exactly as written.

```
1 #kbd("cmd", "shift", "P")
2 #kbd("up") #kbd("down") #kbd("left") #kbd("right")
```



With `layout: "mac"` every common key name maps to its Apple glyph (`ctrl` → `⌘`, `shift` → `⇧`, `enter` → `↵`, `esc` → `⌫`, ...).

```
1 #let mac = keyle.config(layout: "mac", delim: none)
2 #mac("cmd+shift+P") #h(1em)
3 #mac("Ctrl", "Alt", "Del") #h(1em)
4 #mac("enter") #mac("esc") #mac("tab")
```



Set `normalize: false` in `config` to switch normalization off.

2.5 HTML Export

Under Typst's HTML export (`typst compile --features html`), the generated renderer adapts automatically; PDF/PNG/SVG output is unaffected. The `html` option of `config` selects the mode:

- "frame" (default): embeds the exact themed rendering as inline SVG via `html.frame`.
- "kbd" : emits semantic `<kbd>` elements, styled by the browser or your CSS.
- none : passes the themed content through unchanged.

```
1 #import "@preview/keyle:0.4.0": kbd
2 Press #kbd("Ctrl+Shift+P") to open the command palette.
3
4 #let sem = keyle.config(html: "kbd")
5 Copy with #sem("Ctrl", "C"). // -> <kbd>Ctrl</kbd>+<kbd>C</kbd>
```

3 Themes

3.1 Built-in Themes

Theme functions are available in the `keyle.themes` dictionary, or by name in `config`.

```
#keyle.config(theme: "standard")
```

Home

```
#keyle.config(theme: "deep-blue")
```

HOME

```
#keyle.config(theme: "type-writer")
```

HOME

```
#keyle.config(theme: "minimal")
```

Home

```
#keyle.config(theme: "radix")
```

Home

```
#keyle.config(theme: "flowbite")
```

Home

```
#keyle.config(theme: "flowbite-dark")
```

Home

```
#keyle.config(theme: "daisy")
```

Home

```
#keyle.config(theme: "biolinum")
```

Home

3.2 Custom Themes

You can create your own theme by defining a function that takes a symbol and returns themed content.

```

1 // https://www.radix-ui.com/themes/playground#kbd
2 #let radix_kdb(content) = box(
3   rect(
4     inset: (x: 0.5em),
5     outset: (y: 0.05em),
6     stroke: rgb("#1c2024") + 0.3pt,
7     radius: 0.35em,
8     fill: rgb("#fcfcfd"),
9     text(fill: black, font: (
10       "Helvetica Neue",
11       "TeX Gyre Heros",
12     ), content),
13   ),
14 )
15 #let kbd = keyle.config(theme: radix_kdb)
16 #kbd("% D") #kbd("^ F")

```

⌘ D ^ F

3.3 Extending a Theme with .with()

Every built-in theme is a `keycap` or `svg-keycap` factory with its style pre-bound. Because they are ordinary functions, you extend any of them using Typst's native `.with(..)`, without learning a bespoke override API. The text layer (`text-args`, `wrap`) and the cap layer (`fill`, `stroke`, `radius`, `raise`, ...) stay separate.

```

1 #let rose = keyle.themes.flowbite.with(
2   fill: rgb("#fee2e2"),
3   stroke: rgb("#fca5a5"),
4   text-args: (fill: rgb("#991b1b"), weight:
5     "bold"),
6 )
7 #let kbd = keyle.config(theme: rose)
8 #kbd("Ctrl", "Shift", "K")

```

Ctrl + Shift + K

4 Symbols

4.1 Mac Keyboard Symbols

KEYLE provides symbols for Mac keyboards. You may need to install the `Fira Code` font to see the symbols correctly.

- [Fira Code @ Google Fonts](#)
- [Apple Mac Keyboard Symbols](#)

```

1 #let mac-key = keyle.mac-key
2 #let kbd = keyle.config(theme: keyle.themes.standard)
3 #let mac-key-font = (
4   "FiraCode Nerd Font Mono"
5 )
6 #grid(
7   columns: (2fr, 1fr, 2fr, 1fr),
8   rows: 2em,
9   align: horizon,
10   ..mac-key.pairs().map(item => (
11     raw("#mac-key." + item.at(0)),
12     kbd(
13       text(font: mac-key-font, item.at(1))
14     ),
15   )).flatten()
16 )

```

#mac-key.command



#mac-key.shift



#mac-key.option



#mac-key.control



#mac-key.return		#mac-key.delete	
#mac-key.forward-delete		#mac-key.escape	
#mac-key.left		#mac-key.right	
#mac-key.up		#mac-key.down	
#mac-key.pageup		#mac-key.pagedown	
#mac-key.home		#mac-key.end	
#mac-key.tab-right		#mac-key.tab-left	

4.2 Linux Biolinum Keyboard Symbols

KEYLE provides symbols for the Linux Biolinum Keyboard font.

- Linux Biolinum Keyboard

```

1  #let biolinum-key = keyle.biolinum-key
2  #let kbd = keyle.config(theme: keyle.themes.biolinum)
3  #grid(
4    columns: (5fr, 3fr, 5fr, 3fr),
5    rows: 2em,
6    align: horizon,
7    ..biolinum-key.pairs().map(item => (
8      raw("#biolinum-key." + item.at(0)),
9      kbd(item.at(1)),
10    )).flatten()
11 )

```

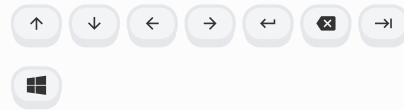
#biolinum-key.Strg		#biolinum-key.Alt	
#biolinum-key.Ctrl		#biolinum-key.Shift	
#biolinum-key.Tab		#biolinum-key.Enter	
#biolinum-key.Capslock		#biolinum-key.Home	
#biolinum-key.Del		#biolinum-key.Ins	
#biolinum-key.End		#biolinum-key.Space	
#biolinum-key.Esc		#biolinum-key.PageUp	
#biolinum-key.PageDown		#biolinum-key.Back	
#biolinum-key.Pad_0		#biolinum-key.Pad_1	
#biolinum-key.Pad_2		#biolinum-key.Pad_3	
#biolinum-key.Pad_4		#biolinum-key.Pad_5	
#biolinum-key.Pad_6		#biolinum-key.Pad_7	
#biolinum-key.Pad_8		#biolinum-key.Pad_9	
#biolinum-key.Pad_Div		#biolinum-key.Pad_Add	
#biolinum-key.Pad_Sub		#biolinum-key.Pad_Mul	
#biolinum-key.Pad_Enter		#biolinum-key.Mac_Cmd	

#biolinum-key.Mac_Opt		#biolinum-key.Mac_FDel	
#biolinum-key.Mac_Del		#biolinum-key.GNU	
#biolinum-key.Win		#biolinum-key.Tux	
#biolinum-key.delim_plus	+	#biolinum-key.delim_minus	-

4.3 SVG Key Glyphs

A key symbol is just content, so non-textual keys can be passed as inline SVG glyphs from `#keyle.svg-key`. Available names are `up`, `down`, `left`, `right`, `enter`, `backspace`, `tab` and `win`.

```
1 #let kbd = keyle.config(theme:
  keyle.themes.flowbite)
2 #kbd(keyle.svg-key.up) #kbd(keyle.svg-key.down)
3 #kbd(keyle.svg-key.left) #kbd(keyle.svg-key.right)
4 #kbd(keyle.svg-key.enter) #kbd(keyle.svg-
  key.backspace)
5 #kbd(keyle.svg-key.tab) #kbd(keyle.svg-key.win)
```



5 Reference

5.1 keyle

- `gen-examples()`
- `resolve-theme()`
- `expand-shortcuts()`
- `normalize-key()`
- `join-keys()`
- `config()`

Variables

- `kbd`

5.1.1 gen-examples

Generate examples for the given keyboard rendering function.

Parameters

`gen-examples` (`kbd`: `function`) -> `content`

kbd `function`

The keyboard rendering function.

5.1.2 resolve-theme

Resolve a theme given by name or by renderer function.

Parameters

`resolve-theme` (`theme`: `str` or `function`) -> `function`

theme `str` or `function`

A key from `themes` (e.g. "flowbite") or a `sym => content` function.

5.1.3 expand-shortcuts

Split shortcut strings on `+`: `"Ctrl+Shift+P" -> ("Ctrl", "Shift", "P")`. A string without `+`, or one that would produce an empty part (e.g. `"+"` itself), is kept as a single key – pass a literal plus as its own argument: `kbd("Ctrl", "+")`.

Parameters

```
expand-shortcuts(keys) -> array
```

5.1.4 normalize-key

Map a key name through an alias table, case-insensitively. Non-string keys (e.g. `svg-key` glyphs) pass through untouched.

Parameters

```
normalize-key(  
  key,  
  aliases  
) -> any
```

5.1.5 join-keys

Join rendered keys with a delimiter between them.

The delimiter is rendered through the theme itself: themes that answer the delimiter handshake (`keycap` / `svg-keycap` / `type-writer` based) render it as if it were a key – same box, baseline and centering – but with an invisible shell, so it aligns with the keys by construction. Themes that don't know the handshake – plain custom functions – fall back to measurement: the cap's descent below the text baseline is recovered by measuring a line that pairs a cap with a zero-width probe of known height.

Parameters

```
join-keys(  
  keys,  
  theme,  
  delim  
) -> content
```

5.1.6 config

Config function to generate a keyboard rendering function.

The generated function accepts any number of keys plus per-call `compact` and `delim` overrides:

```
kbd("Ctrl", "Shift", "P"), kbd("Ctrl+Shift+P"), kbd("cmd", "K") .
```

Parameters

```
config(  
  theme: str function,  
  compact: bool,  
  delim: str content,  
  layout: auto str,  
  normalize: bool,  
  parse: bool,  
  html: str none  
) -> function
```

theme (str) or (function)

The theme to use: a name from `themes` (e.g. "flowbite"), any preset optionally extended with `.with(...)` , or a custom `sym => content` function.

Default: `themes.standard`

compact (bool)

Whether to render keys in a compact format.

Default: `false`

delim (str) or (content)

The delimiter to use between keys.

Default: `"+"`

layout (auto) or (str)

Key-name normalization layout. `auto` maps only unambiguous aliases (`"cmd"` -> ⌘, `"up"` -> ↑, ...); `"mac"` additionally maps `"ctrl"` / `"shift"` / `"enter"` /... to their Apple glyphs.

Default: `auto`

normalize (bool)

Whether to map key-name aliases at all. Set to `false` to render every key name exactly as written.

Default: `true`

parse (bool)

Whether to split shortcut strings such as `"Ctrl+Shift+P"` into individual keys.

Default: `true`

html (str) or (none)

Output mode under HTML export (`typst compile --features html`): `"frame"` embeds the exact themed rendering as inline SVG, `"kbd"` emits semantic `<kbd>` elements styled by the browser, `none` outputs the raw themed content unchanged. PDF/PNG/SVG export is unaffected.

Default: `"frame"`

5.1.7 kbd (function)

Ready-to-use keyboard renderer with default settings, so that `#import "@preview/keyle:0.4.0": kbd` works out of the box. Use `config(...)` for a customized renderer.

5.2 cap

- `style-text()`

- `keycap()`
- `svg-keycap()`

5.2.1 style-text

Text layer: turn a key symbol into styled content.

`wrap` may be any `content -> content` function, so builtin transforms can be passed directly, e.g. `wrap: smallcaps` or `wrap: upper` .

Parameters

```
style-text(
  sym: any,
  args: dictionary,
  wrap: function
) -> content
```

sym any

The key symbol.

args dictionary

Arguments spread into `text` .

Default: `(:)`

wrap function

Content transform applied after styling.

Default: `it => it`

5.2.2 keycap

Keycap factory using vector rectangles (the rect backend).

Bind styling with `.with(...)` to obtain a theme renderer `sym => content` .

Parameters

```
keycap(
  fill: color,
  stroke: stroke,
  radius: length ratio,
  inset: length dictionary,
  raise: length,
  shadow: color none,
  baseline: length ratio,
  text-args: dictionary,
  wrap: function,
  sym: any
) -> content
```

fill (color)

Face fill color.

Default: `rgb( "#eeeeee" )`

stroke (stroke)

Face border.

Default: `rgb( "#555555" ) + 0.6pt`

radius (length) or (ratio)

Corner radius.

Default: `2pt`

inset (length) or (dictionary)

Inner padding around the symbol.

Default: (x: `3pt`, y: `2.5pt`)

raise (length)

Diagonal drop-shadow extent; `0pt` renders a flat cap.

Default: `1.2pt`

shadow (color) or (none)

Shadow color, or `none` to disable.

Default: `rgb( "#555555" )`

baseline (length) or (ratio)

Baseline shift of the cap relative to surrounding text.

Default: `0.1em`

text-args (dictionary)

Text layer arguments spread into `text`.

Default: (fill: `black`)

wrap (function)

Text layer content transform.

Default: `it => it`

sym any

The key symbol.

5.2.3 svg-keycap

Keycap factory using a generated SVG shell (the svg backend).

The SVG is stretched to the measured symbol size so corners never distort. Bind styling with `.with(...)` to obtain a theme renderer `sym => content`.

Parameters

```
svg-keycap(  
  fill: (color),  
  stroke: (color),  
  stroke-width: (float),  
  radius: (float),  
  lip: (float),  
  shadow: (color) (auto) (none),  
  pad: (dictionary),  
  baseline: (length) (ratio),  
  text-args: (dictionary),  
  wrap: (function),  
  sym: any  
) -> (content)
```

fill (color)

Face fill color.

Default: `rgb("#f3f4f6")`

stroke (color)

Face border color.

Default: `rgb("#d1d5db")`

stroke-width (float)

Border width in points.

Default: `1.2`

radius (float)

Corner radius in points.

Default: `6`

lip `{ float }`

Bottom shadow lip height in points.

Default: `3`

shadow `{ color } or { auto } or { none }`

Shadow color, `auto` to derive from `stroke`, or `none`.

Default: `auto`

pad `{ dictionary }`

Inner padding around the symbol.

Default: (x: `6pt`, y: `3.5pt`)

baseline `{ length } or { ratio }`

Baseline shift of the cap relative to surrounding text.

Default: `0.3em`

text-args `{ dictionary }`

Text layer arguments spread into `text`.

Default: (fill: `rgb("#1f2937")`, weight: `"semibold"`)

wrap `{ function }`

Text layer content transform.

Default: `it => it`

sym `{ any }`

The key symbol.